



L'HÉRITAGE

Généralisation-Spécialisation

Super Classe de
Client et Garagiste

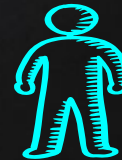

class Person

HÉRITAGE



class Client

HÉRITAGE



class Garagiste

Sous-classe
Ou
Classe dérivée
De
Person

 A SAVOIR !

- Hérite des propriétés
- Hérite des méthodes



LES AVANTAGES DE L'HÉRITAGE

L'héritage permet de :

- Récupérer le comportement standard d'une classe et de **réutiliser les propriétés et des méthodes définies dans la super classe**. (cf. TP 4)
- **Ajouter des fonctionnalités supplémentaires** en créant de nouvelles propriétés et de nouvelles méthodes dans la classe dérivée. (cf. TP 4)
- **Modifier le comportement standard d'une classe** en remplaçant certaines méthodes de la classe parente dans la classe dérivée. (cf. TP 5)

C'est un gain de temps dans les tâches de codage :

- Le programmeur dispose d'une bibliothèque d'objets standards qu'il peut **adapter pour les besoins de son application**.
- Pour la maintenance des applications, il est possible de **corriger un comportement anormal d'une classe en dérivant cette classe** et en subsistant les méthodes boguées. Il n'est même pas nécessaire d'avoir les sources.

HÉRITAGE



SUPER-CLASSE

```
public class ClasseParente {  
  
    private int PropParentPrivate;  
    protected int PropParentProtected;  
    public int PropParentPublic;  
  
    private void MethodeParentPrivate() {  
        Console.WriteLine ("Méthode privée parent");  
    }  
  
    protected void MethodeParentProtected() {  
        Console.WriteLine("Méthode protected parent");  
    }  
  
    public void MethodeParentPublic() {  
        Console.WriteLine("Méthode publique parent");  
    }  
}
```

SOUS-CLASSE

```
public class ClasseFille : ClasseParente {  
  
    private int PropFillePrivate;  
  
    public void MethodeFillePublic() {  
  
        Console.WriteLine("Méthode spécifique ClasseFille");  
  
        Console.WriteLine(PropFillePrivate);  
  
        Console.WriteLine(PropParentProtected);  
  
        Console.WriteLine(PropParentPublic);  
  
        MethodeParentProtected();  
  
        MethodeParentPublic();  
    }  
}
```



CONSTRUCTEURS DANS UNE CLASSE DÉRIVÉE

```
class Garagiste : Personne
{
    // Niveau de compétence sur 100: Taux de réussite de la réparation
    private ushort _Skill;
    public ushort Skill
    {
        get { return _Skill; }
        set { _Skill = (value > 100) ? 100 : value; }
    }

    public Garagiste (string nom, string prenom, int age, ushort skill)
    // Appel du constructeur Parent Obligatoire
    : base(nom, prenom, age)
    {
        this.Skill = skill;
    }
}
```

Le constructeur de la classe dérivée doit obligatoirement appeler un constructeur de la classe Parente.

base() appelle le constructeur correspondant de la classe Parente en lui passant en paramètres les valeurs nécessaires.

Notez qu'une classe déclarée **sealed** est une classe finale, elle **ne pourra pas être dérivée**.

Dans le framework .net, c'est le cas de la classe .

ENNONCÉ

1. Créer deux nouvelles classes Entreprise et Garage. Garage Hérite de Entreprise.
2. Créer deux nouvelles classes Client et Garagiste. Garagiste et Client héritent de Person.
3. Une entreprise comprend un numéro SIRET et peut afficher son identité
4. Un garage gère la liste des voitures à réparer et peut assigner un garagiste à l'une d'entre elles.
5. Un Client peut amener ou récupérer sa voiture au garage.



TP 4



UTILISER

- Héritage
- Utilisation de **base**
- Classe
- Méthodes
- Accesseurs